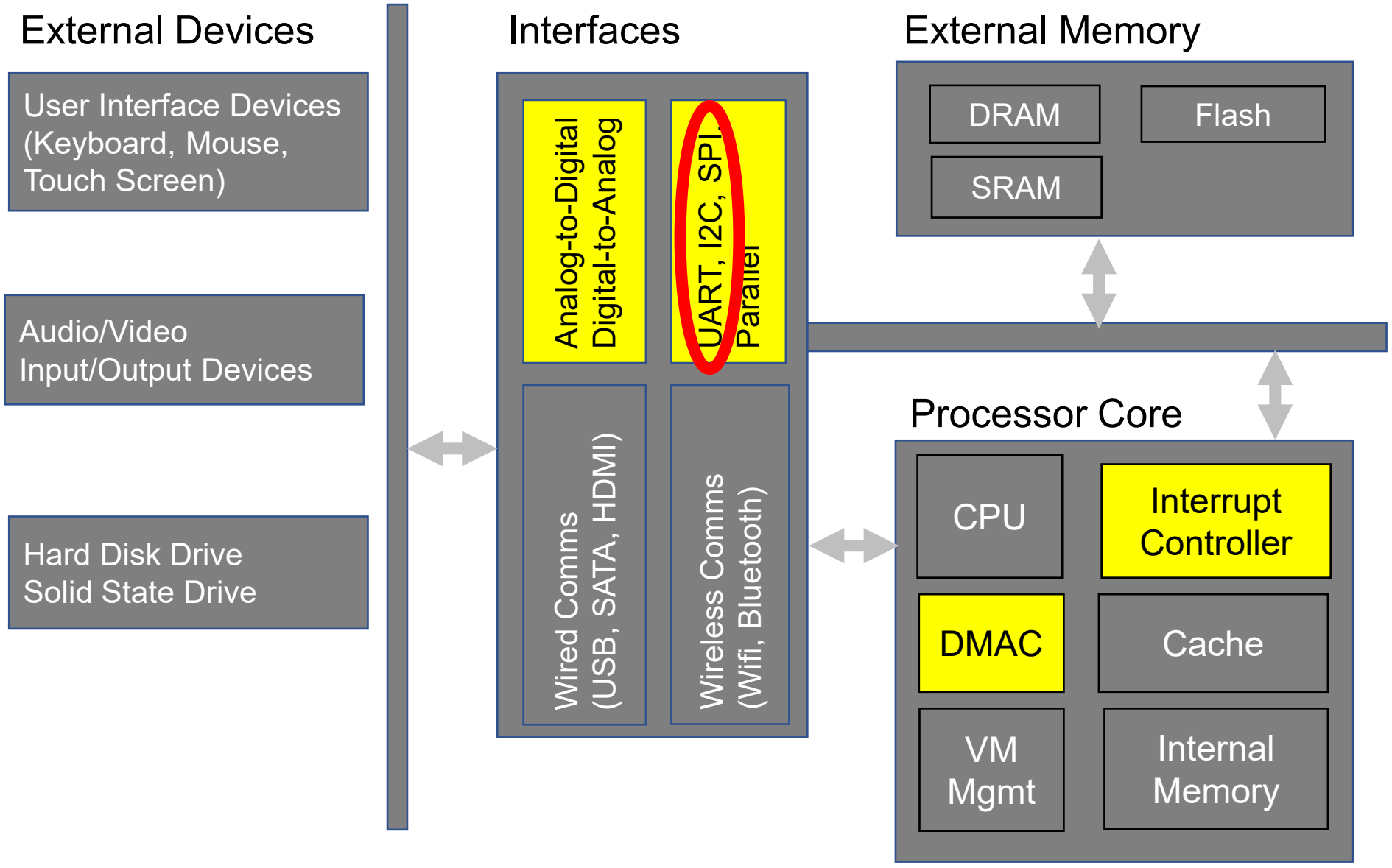
SC1006 Computer Organisation and Architecture

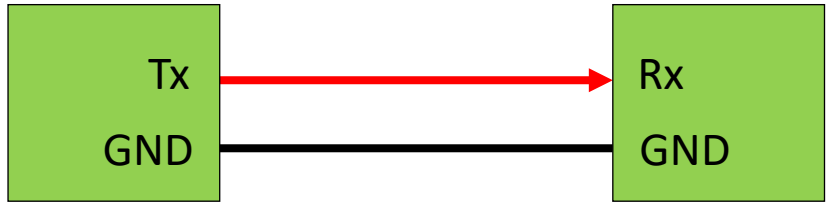
Serial Data Transfer

Prof Lin WeiSi
College of Computing and Data Science, NTU
Email: wslin@ntu.edu.sg

Computer System Block Diagram



Serial Data Transfer



- Data is transferred one bit at a time over a single data line. Comparatively, parallel data interface transfer multiple bits simultaneously.
- Less affected by signal skew and crosstalk because there are less electrical wires involved compared to parallel data transfer. Hence, able to support higher frequency clocking.
- Data transfer rate lower (compared to parallel interface) given the same clock rate since only one data line is available.

Serial Data Transfer Pros and Cons

- Advantage
  - Less affected by signal skew and crosstalk because there are less electrical wires involved compared to parallel data transfer. Hence, able to support higher frequency clocking.
  - Able to transfer data reliably over a longer distance.
  - Lower cost since less wires and connectors are needed.
- Disadvantage
  - Data transfer rate lower given the same clock rate since only one data line is available.
  - Hardware interface design typically more complex as it need to handle serial to parallel conversion (a processor typically only process in bytes or multiple bytes).

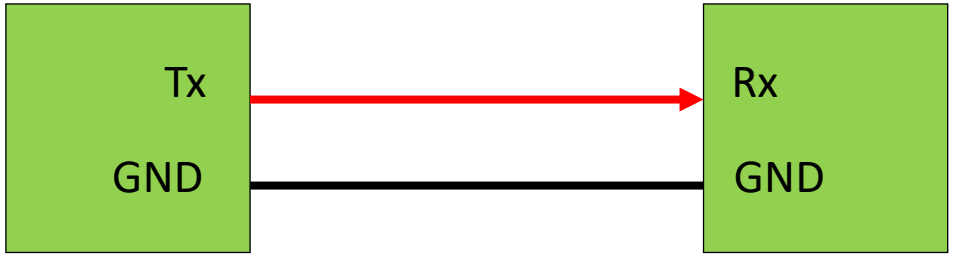
Classification of Serial Data Transfers

According to

- **Directions of transfers**
- **Methods of synchronization**

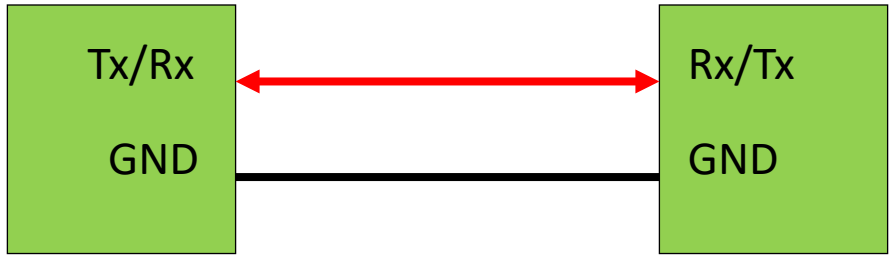
Serial Data Transfer Mode

- Simplex: Data transfer in one direction only, with 1 data line.



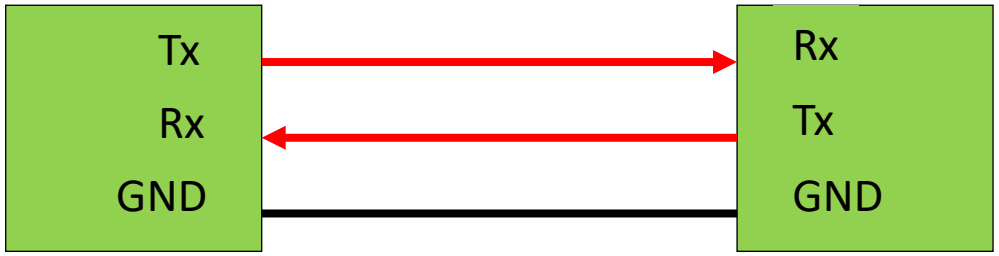
Serial Data Transfer Mode

- Half-Duplex: Data transfer in both directions, but RX and TX is mutually exclusive, also with 1 data line.



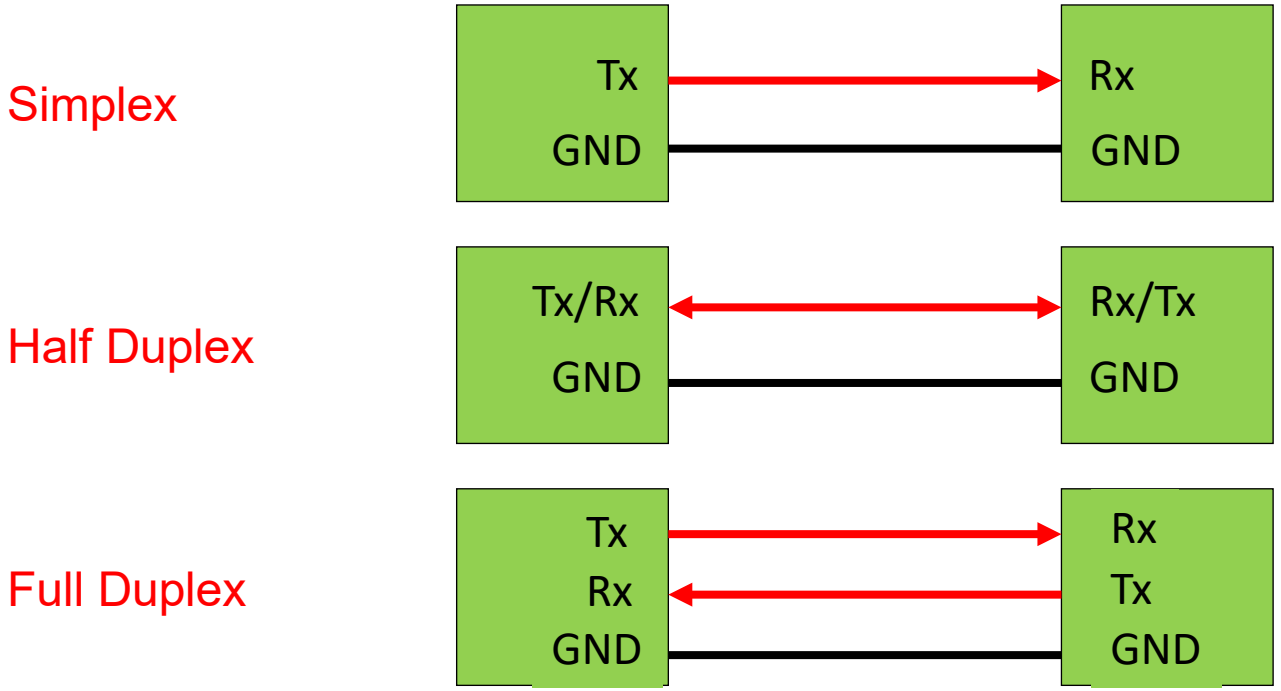
Serial Data Transfer Mode

- Full Duplex: Simultaneous RX and TX, with 2 data lines.



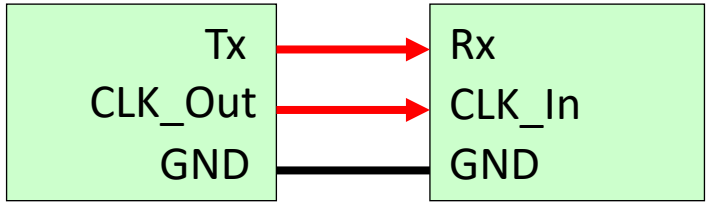
Serial Data Transfer Mode

- Simplex: Data transfer in one direction only.
- Half-Duplex: Data transfer in both direction, but RX and TX is mutually exclusive.
- Full Duplex: Simultaneous RX and TX



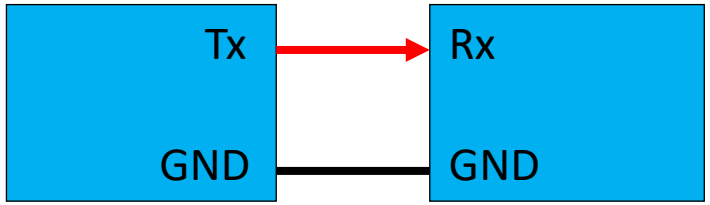
Synchronous vs Asynchronous

Synchronous (similar to parallel interfaces)



- In synchronous transmission, there is a common clock signal to synchronize the data transfer. E.g. receiver to latch in the data at every rising edge of the clock.

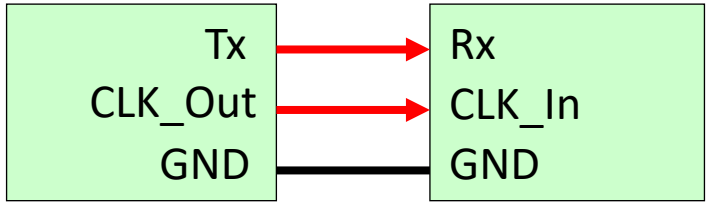
Asynchronous (not available in parallel interfaces)



- In asynchronous transmission, there is no common clock signal so devices have to agree on a pre-fixed clock frequency to use for data transfer.

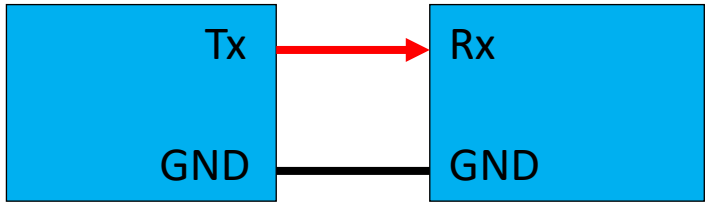
Synchronous vs Asynchronous

Synchronous (similar to parallel interfaces)



- In synchronous transmission, there is a common clock signal to synchronize the data transfer. E.g. receiver to latch in the data at every rising edge of the clock.

Asynchronous (not available in parallel interfaces)



- In asynchronous transmission, there is no common clock signal so devices have to agree on a pre-fixed clock frequency to use for data transfer.

SC1006 Computer Organisation and Architecture

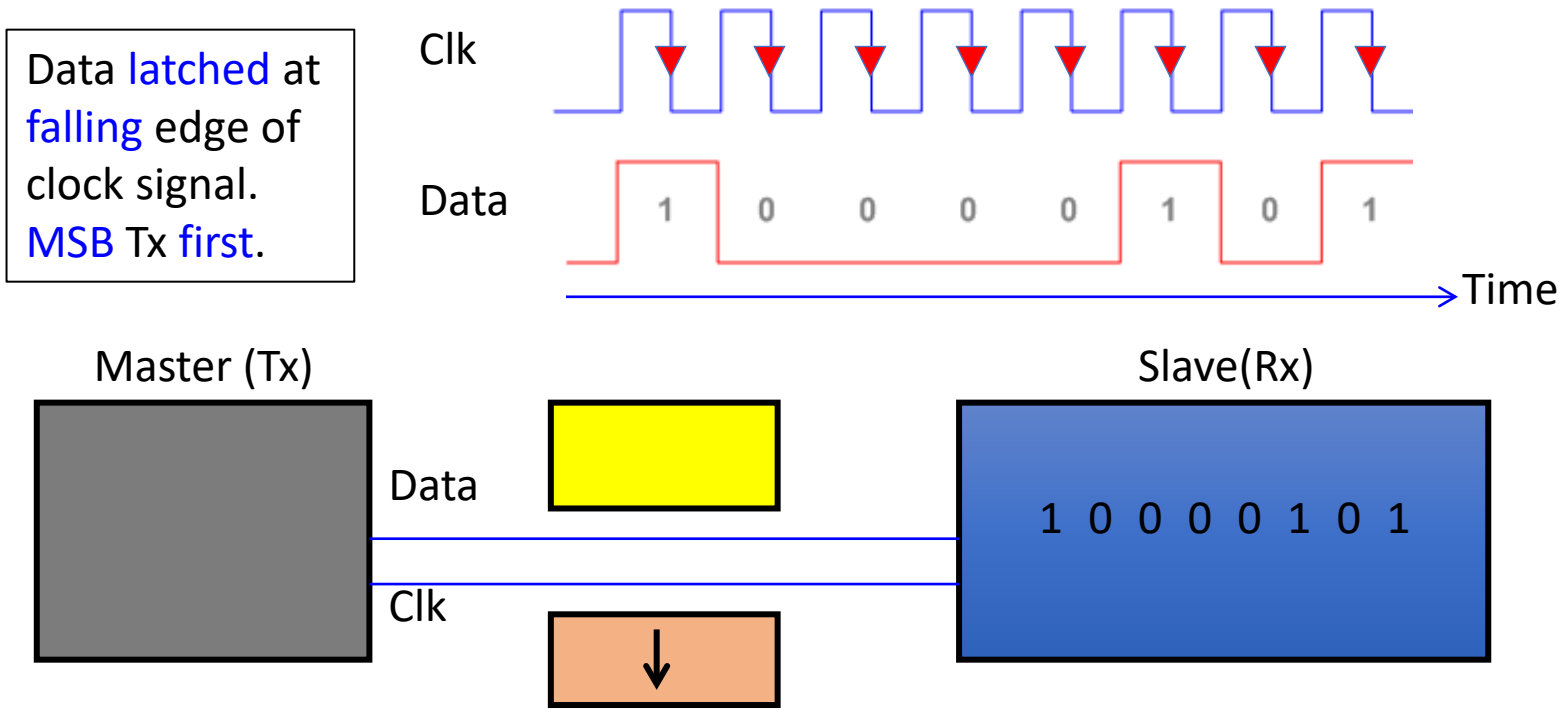
Synchronous Serial Data Transfer

Prof Lin WeiSi
College of Computing and Data Science, NTU
Email: wslin@ntu.edu.sg

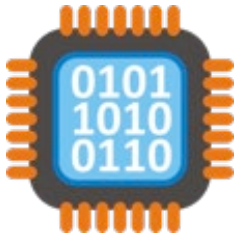
Synchronous Serial Transfer

- Common clock signal between transmitter and receiver to synchronize the data transfer.
- Master-Slave configuration. With Master providing the clock signal.
- Potentially allows faster transfer rate since no data overhead is needed to synchronize the transfer.

Data latched at falling edge of clock signal. MSB Tx first.



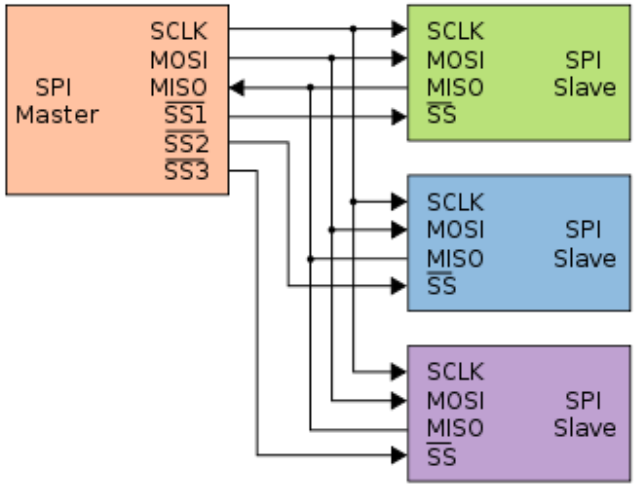
I2C and SPI Bus

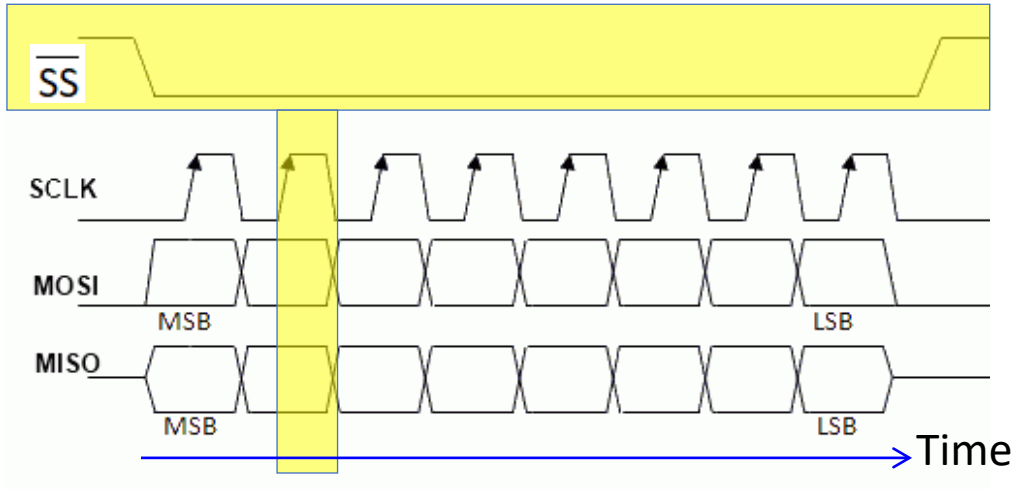


- Popular serial buses used by processor to transfer data and control many peripheral devices.
  - Accelerometers
  - Temperature Sensors
  - Touch Screen Controllers
  - Power Supply Modules Configuration
  - Audio/Video Codecs Configuration

I2C will not be covered in this module

Serial Peripheral Interface (SPI) Bus





- To start the transfer
  - Slave Select (SS) has to be pulled Low.
- Data transfer
  - MOSI: Master-Out-Slave-In (Master Output, Slave Input)
  - MISO: Master-In-Slave-Out (Slave Output, Master Input)
  - Data on MOSI and MISO latched in on rising/falling clock edge (configurable)
- Allow multipe slaves via use of multiple Slave Select.

SC1006 Computer Organisation and Architecture

Asynchronous Serial Data Transfer

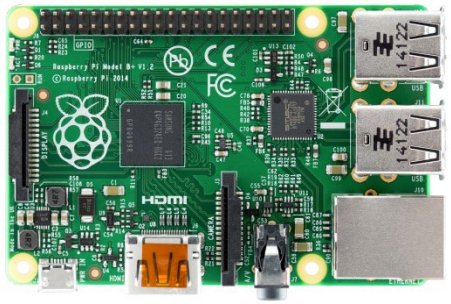
Prof Lin WeiSi
College of Computing and Data Science, NTU
Email: wslin@ntu.edu.sg

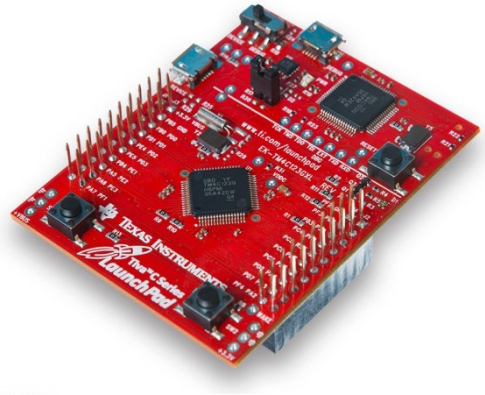
Asynchronous Serial Transfer

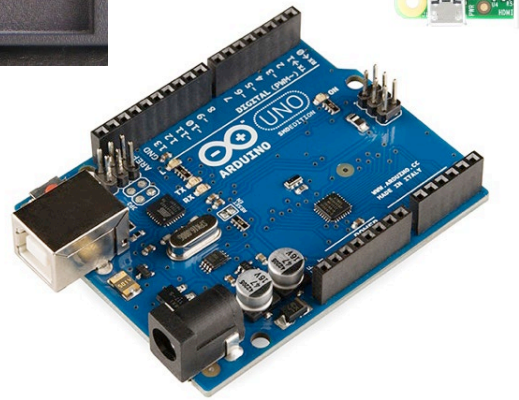
- No common clock is provided between transmitter and receiver.
- Prior to the transmission, the receiver needs to know the transmitting clock rate and the number of bits that are to be transferred with each data packet.
- Special SYNC words are used to indicate START/STOP condition.
- Upon receiving the START SYNC Word, the receiver then uses its own local clock to track the timing.
- Potential skew issue between the two local clocks as transmission progresses.
- Asynchronous Transmission typically also uses SYNC word/bits to provide occasional time stamp for receiver to synchronize its clock to the transmitter clock (helps to reduce clock skew between the two clocks).

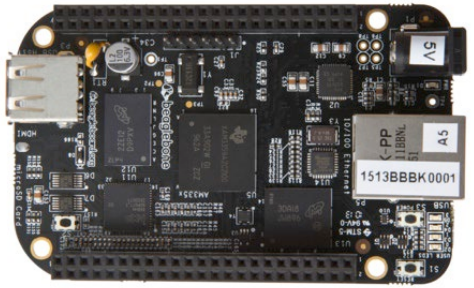
Universal Asynchronous Receiver Transmitter (UART)

- One of the most commonly used serial interfaces.
  - PC Serial COM Port (RS232) uses UART protocol.
  - USB devices uses a Virtual COM Port implementation to connect to the PC.
  - Communication interface between PC and many processor development boards e.g. Arduino Board, TIVA-C Launchpad etc



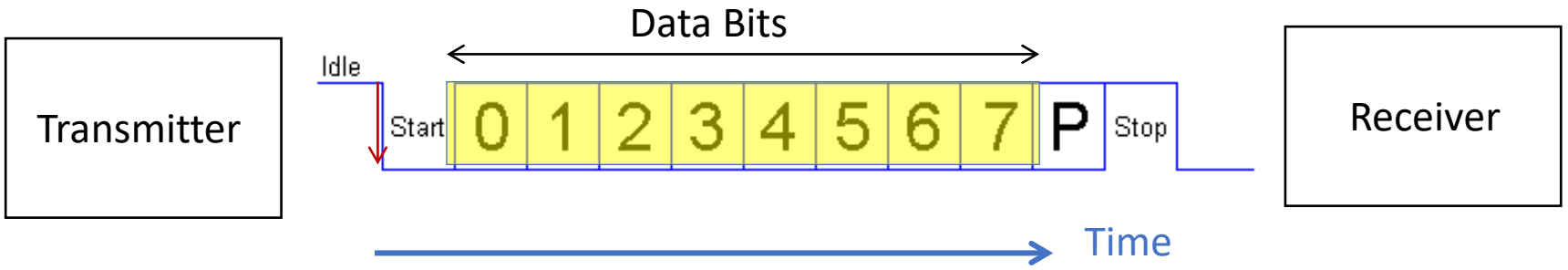






UART Transmit

- During **IDLE** State, the data line is in 'powered' state i.e. Logic '1'.
- The transmitter send a **START** pattern (logic '0') to alert the receiver.
- Sending a logic '0' from idle state (logic '1') will result in a falling edge on the data line. This is typically used by the receiver to detect the start of transmission.
- This is followed by the actual **DATA** at a frequency known to the receiver. The transmitting clock rate is also known as the **baud rate**, and determines the number of bits transmitted per second.
- A **PARITY** bit (optional) may also be sent for the receiver to check the integrity of the data packet.
- A **STOP** bit (Logic '1') terminates the transmission.



Parity Bit

- If parity scheme is enabled, the receiver will also sample the parity bit and checks for parity error.
- If even parity scheme is used, there should be an even number of ‘1’s in the data field and the parity field.
- Hence, if there is an odd number of ‘1’s in the data field, then the parity bit transmitted should be a ‘1’ to make the total even.
- If receiver receives odd number of ‘1’s in a even parity scheme, it’ll flag a Parity Error, meaning one or more bits in the transmission is wrong.
- Vice versa for odd parity scheme.
- The receiver then samples the **STOP** bit(s), if a ‘0’ is detected instead of ‘1’, then receiver will flag a Framing Error.

UART Tx Example

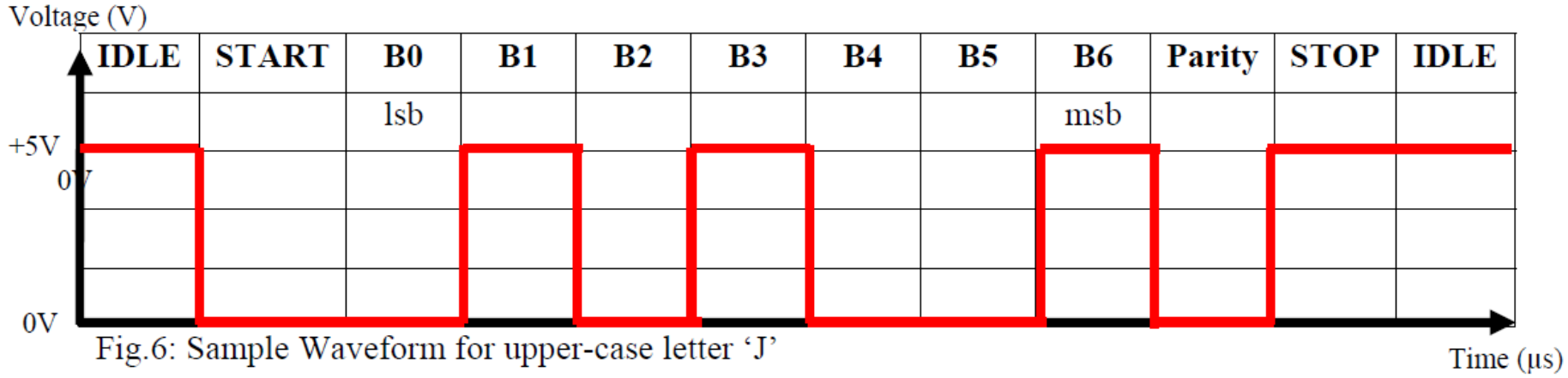
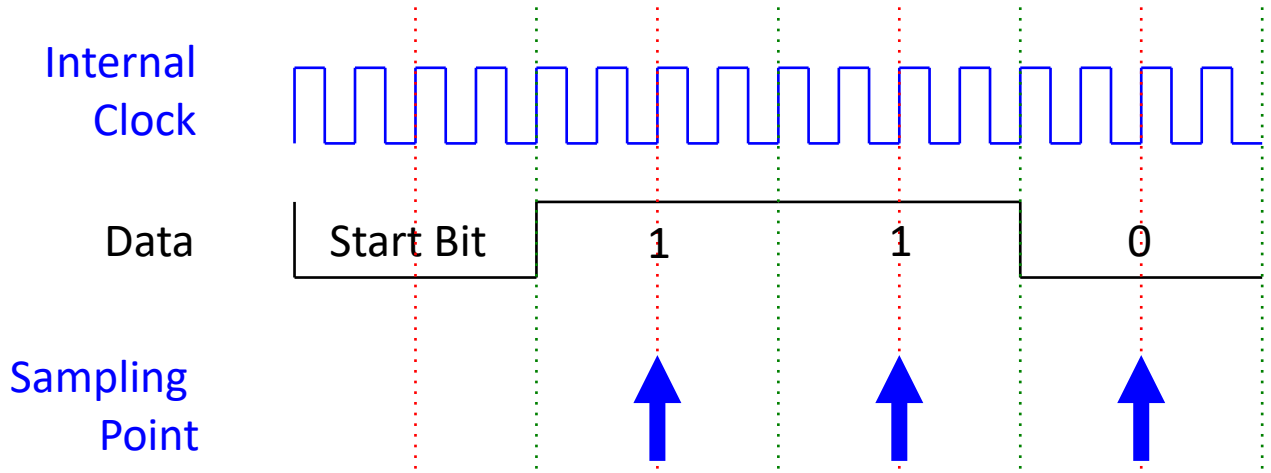


Fig.6: Sample Waveform for upper-case letter 'J'

- Figure 6 above correspond to the configuration 7O1 (7 Data, 1 STOP and Odd parity).
- Capital Letter 'J' => Ascii value = 0x4A
- 0x4A => 100 1010 (binary)
- Presented in LSB first => 0101 001
- IDLE=1, START=0, STOP=1.
- Parity bit logic depends on number of 1's in Data Field and the Parity Scheme (Odd or Even) used. It can be of **N**o (N) parity check.
- Can practise with 8E1, 7N1, 7N2, …

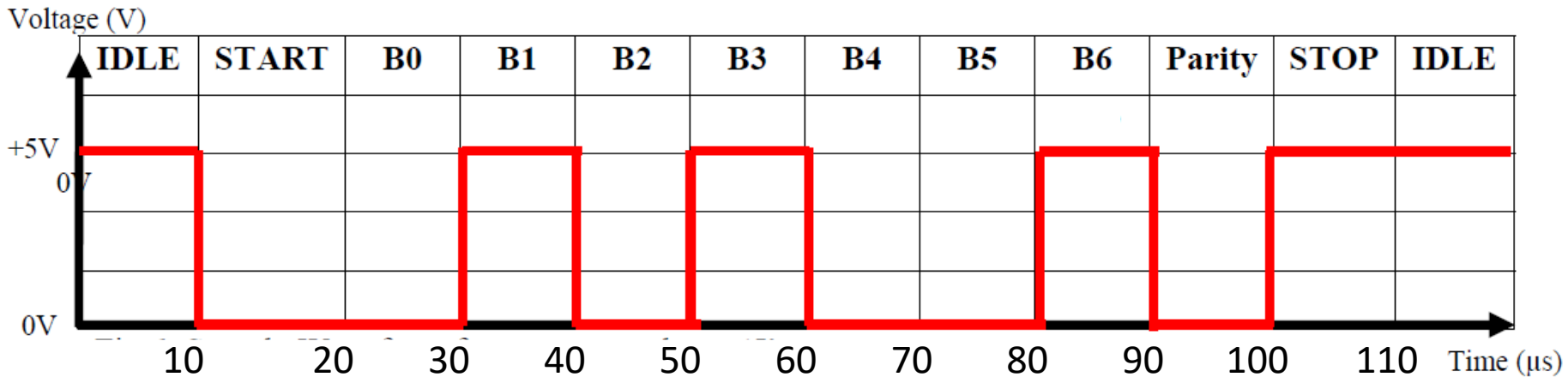
UART Receive

- Receiver monitors the Data line for the Start Bit. In real world design, the falling edge on the data line will trigger an interrupt in the microprocessor to start the receiving process.
- Needs to know the baud rate in order to sample the data bits correctly.
- Internal clock of the UART typically run at a multiple times (e.g. 16X) of the baud rate so as to time the sampling close to the middle of each data bit.
- Below is an example of UART receive with internal clock running at 4X baud rate (for illustration only). Internal clock rate in this example is faster than 4X baud rate in real world implementation.



UART Rx Example

Tx baud rate = 1/(10 us) = 100000 bps. Configuration = 7O1.



- RX bits = 0101001b (actual Data=1001010b)
- RX bits = 0011000b always?
- **Possible if Information Sampled @ 200000 bps:**
  0000110011000011011
  start
  0101001b